

ORACLE DATA INTEGRATION

METADATA INTEGRATION

Product Design • Implementation • Training • Technical Reference • Troubleshooting

Purpose

Advanced-student lab guide for designing, validating, and supporting Oracle Data Integration Metadata Integration.

Metadata Integration lifecycle



Metadata Integration creates a governed path for moving source structures into trusted target metadata. Good design improves consistency, traceability, support readiness, and confidence in downstream planning and reporting structures.

Audience

Oracle EPM implementers, integration developers, solution architects, administrators, support engineers, business reviewers, and advanced learners.

Lab outcome

Learners can design metadata source-to-target synchronization, explain structural validation controls, and diagnose common metadata integration issues across Oracle Data Integration processes.

1. Metadata Integration - Concept and Product Design

Metadata Integration in Oracle Data Integration is the controlled process of bringing structural information such as dimensions, members, hierarchies, aliases, and attributes from source systems into target applications. It ensures that business structures remain consistent across planning, reporting, and downstream operational uses. Good metadata integration is not only a technical sync process; it is also a governance layer that protects structure quality and business meaning.

Metadata area	Responsibility	Avoid placing here
Source Discovery	Identify authoritative metadata sources and scope.	Unapproved shadow data sources.
Mapping & Standardization	Align names, types, parents, and attributes.	One-off manual fixes with no traceability.
Validation	Check structural correctness and policy compliance.	Publishing members before hierarchy checks.
Publish / Sync	Move approved metadata to target applications.	Uncontrolled changes directly in production.

Design principle

Treat metadata integration as a governed business process. The objective is not only to move structures, but to maintain trusted hierarchies, meaningful attributes, and controlled synchronization across the application landscape.

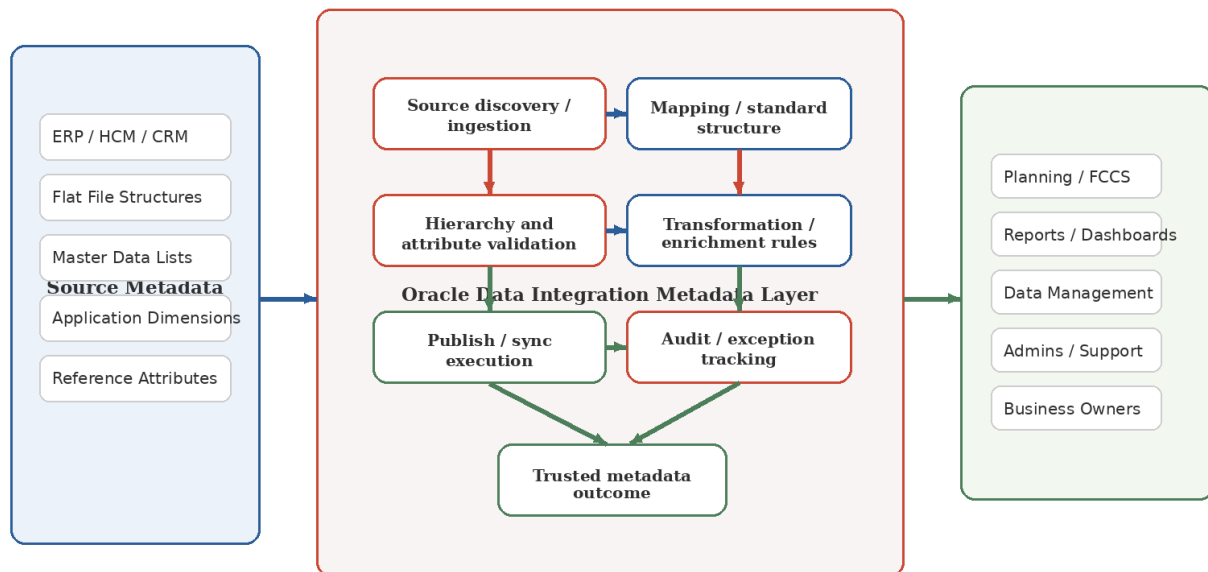
Key capabilities

- Metadata source discovery
- Member and hierarchy synchronization
- Attribute and alias management
- Structural validation and exception handling
- Controlled publish with audit evidence

2. Architecture Overview

The architecture below shows how metadata ingestion, mapping, validation, transformation, and publish activity operate together in Oracle Data Integration.

Reference architecture - Metadata Integration



Metadata Integration combines ingestion, validation, transformation, synchronization, and governance so application structures remain consistent, auditable,

Architecture note

A strong design separates source ingestion, structural rules, publish execution, and exception management. This separation improves support diagnostics and gives administrators clear control over metadata change movement.

3. Core Terminology and Metadata Objects

A shared vocabulary helps project teams, administrators, and advanced learners discuss structural behavior consistently.

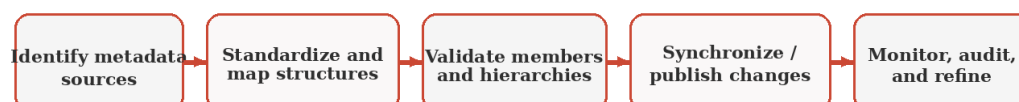
Term	Meaning	Example
Member	Individual metadata object within a dimension	Entity US001
Hierarchy	Parent-child arrangement of members	Total Region > East > NY
Alias	Alternative display name	US001 = United States East
Attribute	Descriptive property attached to a member	Department Type = Cost Center
Publish Run	Controlled synchronization cycle	Weekly Entity Sync
Exception Queue	Failed or warning structural items requiring review	Member parent not found

Typical metadata integration patterns	When to use
Simple metadata refresh	Structures are stable and only periodic updates are needed.
Standard mapped synchronization	Sources differ from target naming and need transformation rules.
Governed structural publish	Frequent hierarchy or attribute changes require validation and approvals.

4. Implementation Process Flow

The following lifecycle provides a repeatable path for designing and implementing supportable metadata integration.

Metadata Integration lifecycle



Step	Owner	Expected output
Define metadata sources, target dimensions, and business purpose	Solution architect / functional lead	Approved metadata strategy.
Design mapping, hierarchy, and attribute rules	Integration developer / architect	Metadata integration specification.
Configure validation, publish, and exception handling	Developer / admin	Configured sync process.
Test additions, updates, deactivations, and edge cases	Developer / tester	Validated test evidence.
Operationalize monitoring and governance	Support lead / business owner	Runbook and production approval.

Implementation tip

Document the journey of one sample member from source to target. Show how its name, parent, alias, and attributes are interpreted, validated, and published.

5. Source and Target Metadata Model

A successful design clearly defines which source structures feed which target dimensions and which fields are mandatory for synchronization.

Design point	Purpose	Examples
Authoritative source	Clarify where member truth originates	ERP cost center master
Target dimension	Specify where metadata will land	Entity, Account, Department
Mandatory fields	Prevent incomplete publishes	Member Code, Parent, Active Flag
Optional descriptors	Enhance search and reporting	Alias, Region, Manager
Change type	Support add, update, move, or deactivate scenarios	Parent change for product line

Model note

If multiple systems contribute to the same metadata domain, define source precedence and conflict resolution rules explicitly before build.

6. Configuration Mockup and Setup Notes

This page acts as a setup reference for the controls teams typically capture when configuring Metadata Integration.

UI field	Example value	Why it matters
Integration Name	MI_ENTITY_WEEKLY_SYNC	Unique support and audit identifier.
Metadata Domain	Entity	Clarifies structural scope.
Source Object	ERP_CC_MASTER	Documents the upstream source.
Target Dimension	Entity	Defines destination structure.
Validation Set	VAL_ENTITY_HIERARCHY_CORE	Controls structure quality.
Publish Mode	Validated changes only	Controls target update behavior.
Owner	Metadata Integration Admin	Defines accountability.

Configuration control

Before production, verify source assumptions, mandatory fields, hierarchy constraints, attribute rules, publish mode, rollback strategy, and evidence retention.

Common mistake

Designing the publish step first and structural validation second. Trusted metadata depends on good parent-child integrity and attribute quality, not only on successful movement.

7. Metadata Load and Sync Patterns

Different integration patterns support different business needs depending on source stability, structural complexity, and governance requirements.

Pattern	What it does	Typical use case
Refresh	Loads latest metadata snapshot into the integration layer	Stable periodic dimension refresh
Delta Sync	Moves only changed members or attributes	Daily maintenance updates
Governed Publish	Publishes validated changes after review or approval	Finance-controlled hierarchies
Exception-driven Sync	Processes clean items and holds problem records	Large domains with occasional data issues

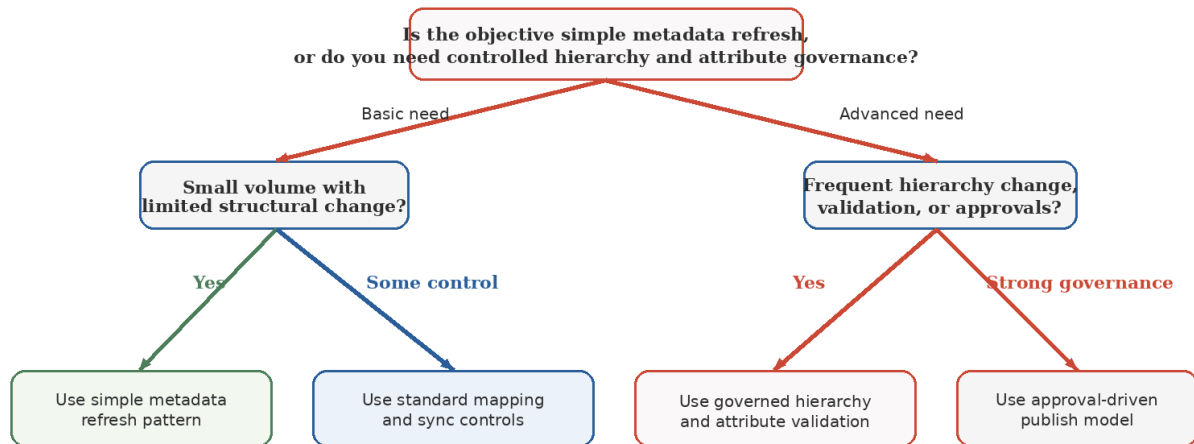
Pattern guidance

Choose a sync pattern based on business risk, not just technical convenience. Higher-risk domains often justify delta controls, validations, and approvals before publish.

8. Decision Tree for Integration Pattern Selection

Projects often ask how much metadata governance is necessary. The decision tree below provides a practical answer path.

Decision tree - What kind of metadata integration is needed?



Guiding rule: metadata integration should balance agility and governance. Choose the lightest design that still protects hierarchy quality, attribute cor

Governance recommendation

Use stronger validation and approval controls whenever metadata changes affect reporting rollups, security, allocations, or planning ownership.

9. Hierarchy and Attribute Management

Metadata Integration becomes especially valuable when it controls both hierarchy quality and descriptive attribute consistency.

Control area	What should be validated	Example
Parent-child integrity	Every member has a valid allowed parent	Department D110 under Region East
Hierarchy depth	Levels conform to design constraints	Product hierarchy max 5 levels
Attribute completeness	Required descriptive fields are populated	Currency attribute not null
Alias consistency	Readable names match business expectations	NY_SLS alias = New York Sales
Status handling	Inactive members are deactivated correctly	Retired cost center blocked from new use

Hierarchy note

A structurally valid member may still be business-invalid if its parent, alias, or attribute values create incorrect rollups or reporting logic.

10. Data-Flow and Control Checkpoints

The control points below show where metadata behavior should be checked during a full synchronization cycle.

Data-flow diagram - metadata from source to trusted publish

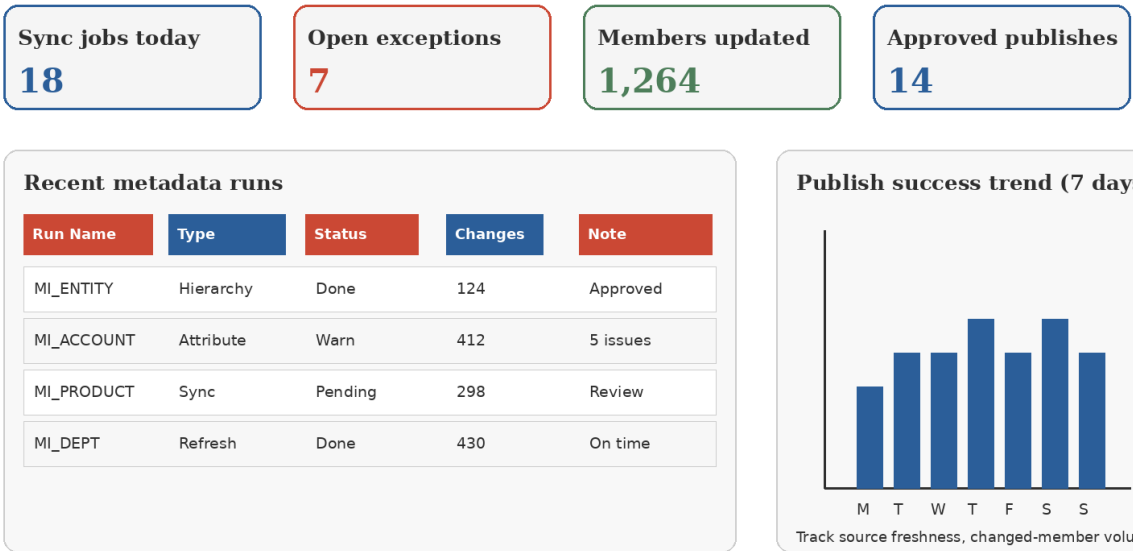


Checkpoint	What to validate	Control evidence
At ingestion	Correct source and record count loaded	Run log and count.
At mapping	Fields and structures align to target design	Mapping validation result.
At structural validation	Hierarchy and attributes pass rules	Validation report.
At publish preparation	Only approved changes are eligible	Preview or exception evidence.
At publish	Target metadata reflects intended changes	Final publish summary.

11. Dashboard Mockup and Monitoring

A metadata dashboard helps operations teams monitor source freshness, changed-member volume, validation results, and publish readiness.

Dashboard mockup - metadata synchronization health



Monitoring KPIs

Track runs executed, members added, updates by domain, hierarchy failures, attribute completeness, rejected changes, publish success rate, and approval turnaround.

12. Change Management and Governance

Metadata changes should be governed because structural shifts can alter rollups, reports, data loads, and operational responsibilities across connected applications.

Governance area	Control question	Example
Change ownership	Who requested and approved the structural update?	Finance master-data owner
Effective timing	When should the metadata become active?	Month-end after close
Impact analysis	Which downstream objects are affected?	Reports, forms, security filters
Rollback readiness	How can the prior state be restored?	Previous hierarchy backup
Audit evidence	How is the publish history preserved?	Run ID + approver note

Governance note

Even a small parent change can materially affect planning aggregation and reporting. Always link metadata publication to ownership and impact review.

13. Technical Reference and Standards

The technical reference below provides practical standards that can be reused in build documents, SOPs, and support runbooks.

Area	Standard / rule	Example
Naming	MI__	MI_ENTITY_WEEKLY_SYNC
Description	State source, structural scope, and publish purpose clearly	Sync entity metadata from ERP to Planning
Testing	Validate add, update, move, deactivate, and publish scenarios	Parent change and inactive member case
Ownership	Capture business and technical owner	Metadata owner + integration admin
Documentation	Maintain source-to-target metadata map	Metadata design register

Metadata to retain for every integration	Purpose
Source definition, target dimension, and key fields	Dependency clarity
Validation rules, exception path, and publish mode	Operational accuracy
Business owner, technical owner, and last review date	Auditability
Run frequency, change log, and rollback notes	Support readiness

14. Log and Error Examples

Metadata issues may appear during ingestion, validation, transformation, or publish. These examples help support teams respond quickly.

Example log / error	Likely cause	Corrective action
Validation failed - Parent member not found for D110	Source hierarchy references unknown parent	Correct source value or load missing parent first.
Publish blocked - Required attribute Currency missing	Mandatory attribute rule not satisfied	Enrich source or default value per policy.
Alias conflict detected for member PROD_410	Two records map to the same alias or member label	Review mapping and alias uniqueness rule.
Run completed but 0 members published	No approved changes or wrong publish filter	Check filter, review status, and source delta.

Support note

Capture integration name, metadata domain, source object, run ID, member count, failed member examples, and exact validation or publish message in every incident ticket.

15. Troubleshooting Handbook

The following flow and checklist help support teams resolve metadata integration issues systematically.

Troubleshooting flow - resolving a metadata integration issue



Symptom	Check first	Escalate when
Members not published	Review validation failures and publish filter	Business deadline is impacted by unresolved structure issues.
Hierarchy incorrect in target	Parent-child mapping and transformation rules	Rollups differ materially from approved design.
Attributes missing or wrong	Source completeness and attribute-mapping logic	Reporting or security depends on those attributes.
PROD differs from TEST	Migration drift, source data differences, or target setup	Root cause is environment-specific and repeatable.

16. Training Lab - Build a Standard Metadata Integration

This lab teaches learners how to configure a metadata integration that loads source structures, validates them, and publishes approved members to a target dimension.

Lab step	Learner task	Expected evidence
1	Define source metadata fields and target dimension.	Short design note.
2	Configure mappings, validation, and publish mode.	Setup evidence.
3	Run the integration on sample metadata.	Execution output and counts.
4	Inspect exceptions and correct invalid members.	Exception evidence.
5	Publish validated result and confirm target update.	Publish evidence.

Lab outcome

Learners should be able to explain the complete path from source metadata through validation and publish to trusted application structure.

17. Training Lab - Advanced Hierarchy and Attribute Scenarios

This advanced lab exposes learners to parent changes, member deactivation, attribute gaps, and approval-driven publication where structural accuracy is essential.

Scenario	What learner should diagnose	Target learning
Parent change causes rollup shift	How hierarchy updates affect downstream behavior	Structural impact awareness
Inactive member still visible	Status handling and publish controls	Lifecycle management
Required attribute missing	Difference between structurally valid and business-valid metadata	Quality discipline
Approved changes not published	How approvals and filters interact	Publish-governance discipline

Advanced exercise

Ask learners to compare a lightweight metadata refresh with an approval-gated metadata publish. Require justification based on business risk, hierarchy importance, and support cost.

18. Implementation Checklist and Best Practices

Use this final page as both a go-live checklist and a summary of metadata integration best practices.

Checklist item	Status
Metadata sources, target domains, and ownership are approved	■
Mapping, hierarchy, attribute, and publish rules are documented	■
Validation, exceptions, and rollback approach are tested	■
Monitoring dashboard and support runbook are attached	■
Business owner, technical owner, and approvers are defined	■
Migration, environment controls, and audit evidence are updated	■

Best practices

- Define authoritative metadata sources clearly.
- Validate parent-child integrity before publish.
- Treat attributes and aliases as governed business data.
- Publish only approved structural changes.
- Keep dashboards and runbooks aligned to integration design.

Document value

This guide can serve as product design documentation, an implementation guide, training material, a technical reference, and a troubleshooting handbook for Oracle Data Integration Metadata Integration.